

Predibag: Predicate-Based Agents

Plan

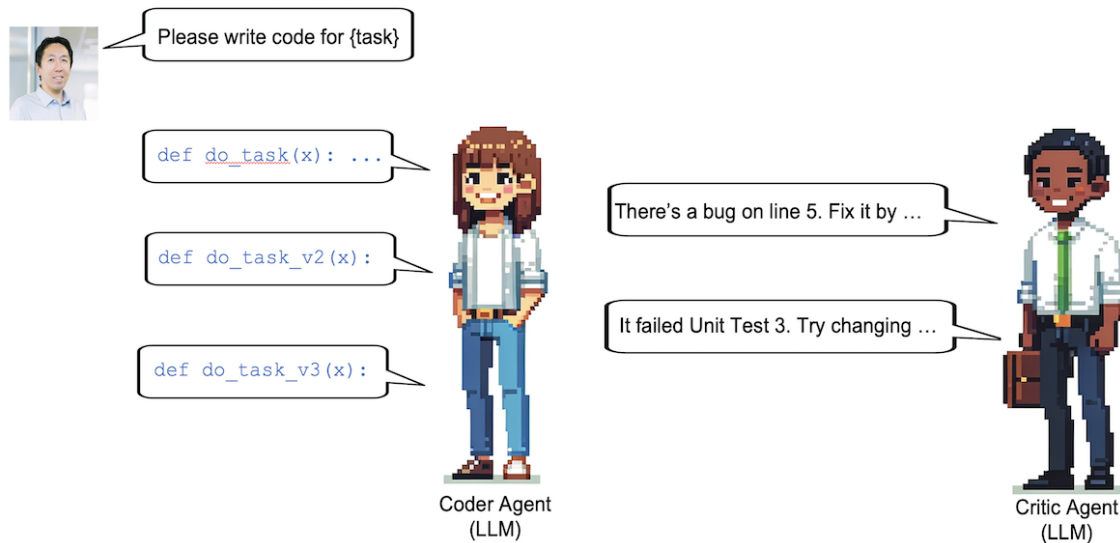
Podcast: <https://notebooklm.google.com/notebook/521cfd7b-dbc0-4ca6-9932-df97980f006a/audio>

- Agents
- Prolog
- Prompting Strategy
- GSM8K Evaluation

Andrew Ng

Experiments show that LLM outputs are improved with agents.

Agentic Design Patterns: Reflection



1. *CRITIC: Large Language Models Can Self- Correct With Tool-Interactive Critiquing (Zhibin Gou et al.)*
2. *SELF-REFINE: Iterative Refinement with Self-Feedback (Aman Madaan et al.)*
3. *Reflexion: Language Agents with Verbal Reinforcement Learning (Noah Shinn et al.)*

Agent behaviors can be broadly categorized into several types:

1. Reactive Behavior:

- Immediate Response: Agents respond to stimuli from the environment
- Reflex Actions: Similar to reflexes in humans, where the response is direct and immediate

2. Goal-Oriented Behavior:

- Objective Pursuit: Agents act with the purpose of achieving specific goals.
- Planning: They may use planning algorithms to determine a sequence of actions that lead to the goal.

3. Learning Behavior:

- Adaptation: Agents improve their performance over time by learning from experience.
- Machine Learning Techniques: Includes supervised, reinforcement and unsupervised learning.

4. Social Behavior

- Cooperation
- Communication
- Negotiation

1. Google

- Vertex AI
- Based on GEMINI

2. ANTHROPIC

- Computer User (API only)
- Based on Claude (not me)

3. OpenAI

- Swarm
- Based on OpenAI model family

4. LangGraph

- Langchain solution
- Any models.

LangGraph

- LangGraph is a library for building stateful, multi-actor applications with LLMs.
- The main use cases for LangGraph are:
 - Conversational agents
 - Long-running, multi-step LLM applications
 - Any LLM application that would benefit from built-in support for persistent checkpoints, cycles and human-in-the-loop interactions

<https://github.com/langchain-ai/langgraph>

```
# Define a new graph
workflow = StateGraph(AgentState, config_schema=GraphConfig)

# Define the two nodes we will cycle between
# They are associated to specific Python functions
workflow.add_node("agent", call_model)
workflow.add_node("action", tool_node)

# Set the entry point as `agent`
# This means that this node is the first one called
workflow.set_entry_point("agent")

# We now add a conditional edge
workflow.add_conditional_edges(
    # First, we define the start node. We use `agent`.
    {
        # If `tools`, then we call the tool node.
        "continue": "action",
        # Otherwise we finish.
        "end": END,
    },
)

# We now add a normal edge from `tools` to `agent`.
workflow.add_edge("action", "agent")
```

What are they trying to do?

↪ Design a DSL grafted onto Python to traverse graphs...

As usual, when people try to reimplement these very old architectures, they come up with the same basic issues:

- ↪ *Backtracking*
- ↪ *State management*
- ↪ *Data management*
- ↪ *Graph pruning*

They need to reimplement an awkward Prolog without the perks....

Prolog: Traversing a Graph

- *Prolog explores a graph of possibilities.*

Knowledge Base:

```
parent("Pierre", "Marie"). Pierre is a parent of Marie  
parent("Jean", "Pierre"). Jean is a parent of Pierre  
parent("Alain", "Jean"). Alain is a parent of Jean
```

- Let's define the *grand-father* relationship...

```
grand_father(?Grandfather, ?Grandson) :-  
    parent(?Grandfather, ?Person),  
    parent(?Person, ?Grandson).
```

Prolog: Traversing a Graph

```
parent("Pierre", "Marie"). Pierre is a parent of Marie  
parent("Jean", "Pierre"). Jean is a parent of Pierre  
parent("Alain", "Jean"). Alain is a parent of Jean
```

- We can then answer questions such as: *Who is the grand-father of Marie?*

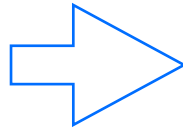
```
grand_father(?GP, "Marie") :-  
    parent(?GP, ?P),  
    parent(?P, "Marie").
```

Prolog: Traversing a Graph

```
parent("Pierre", "Marie"). Pierre is a parent of Marie  
parent("Jean", "Pierre"). Jean is a parent of Pierre  
parent("Alain", "Jean"). Alain is a parent of Jean
```

- We can then answer questions such as: *Who is the grand-father of Marie?*

```
grand_father(?GP, "Marie") :-  
    parent(?GP, ?P),  
    parent(?P, "Marie").
```



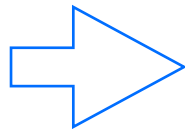
```
grand_father(?GP, "Marie") :-  
    parent(?GP, "Pierre"),  
    parent("Pierre", "Marie").
```

Prolog: Traversing a Graph

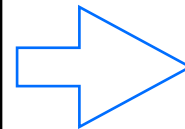
parent("Pierre", "Marie"). *Pierre is a parent of Marie*
parent("Jean", "Pierre"). *Jean is a parent of Pierre*
parent("Alain", "Jean"). *Alain is a parent of Jean*

- We can then answer questions such as: *Who is the grand-father of Marie?*

grand_father(?GP, "Marie") :-
parent(?GP, ?P),
parent(?P, "Marie").



grand_father(?GP, "Marie") :-
parent(?GP, "Pierre"),
parent("Pierre", "Marie").



grand_father("Jean", "Marie") :-
parent("Jean", "Pierre"),
parent("Pierre", "Marie").

Prolog: Traversing a Graph II

```
parent("Pierre", "Marie"). Pierre is a parent of Marie  
parent("Jean", "Pierre"). Jean is a parent of Pierre  
parent("Alain", "Jean"). Alain is a parent of Jean
```

- We can then answer questions such as: *Who are the grand-fathers ?*

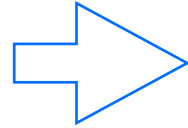
```
grand_father(?GP, ?GS) :-  
    parent(?GP, ?P),  
    parent(?P, ?GS).
```

Prolog: Traversing a Graph II

```
parent("Pierre", "Marie"). Pierre is a parent of Marie  
parent("Jean", "Pierre"). Jean is a parent of Pierre  
parent("Alain", "Jean"). Alain is a parent of Jean
```

- We can then answer questions such as: *Who are the grand-fathers ?*

```
grand_father(?GP, ?GS) :-  
    parent(?GP, ?P),  
    parent(?P, ?GS).
```



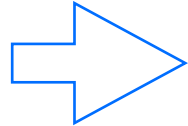
```
grand_father(?GP, "Marie") :-  
    parent(?GP, "Pierre"),  
    parent("Pierre", "Marie").
```

Prolog: Traversing a Graph II

parent("Pierre", "Marie"). *Pierre is a parent of Marie*
parent("Jean", "Pierre"). *Jean is a parent of Pierre*
parent("Alain", "Jean"). *Alain is a parent of Jean*

- We can then answer questions such as: *Who are the grand-fathers ?*

```
grand_father(?GP, ?GS) :-  
    parent(?GP, ?P),  
    parent(?P, ?GS).
```



```
grand_father(?GP, "Marie") :-  
    parent(?GP, "Pierre"),  
    parent("Pierre", "Marie").
```

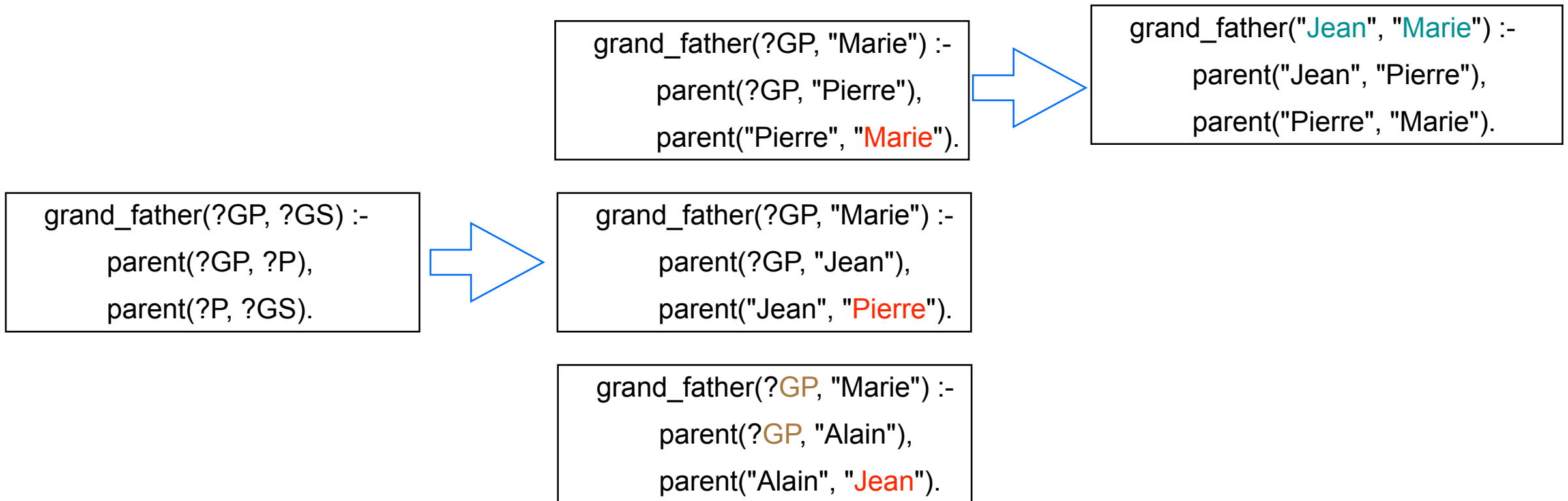
```
grand_father(?GP, "Marie") :-  
    parent(?GP, "Jean"),  
    parent("Jean", "Pierre").
```

```
grand_father(?GP, "Marie") :-  
    parent(?GP, "Alain"),  
    parent("Alain", "Jean").
```

Prolog: Traversing a Graph II

```
parent("Pierre", "Marie"). Pierre is a parent of Marie
parent("Jean", "Pierre"). Jean is a parent of Pierre
parent("Alain", "Jean"). Alain is a parent of Jean
```

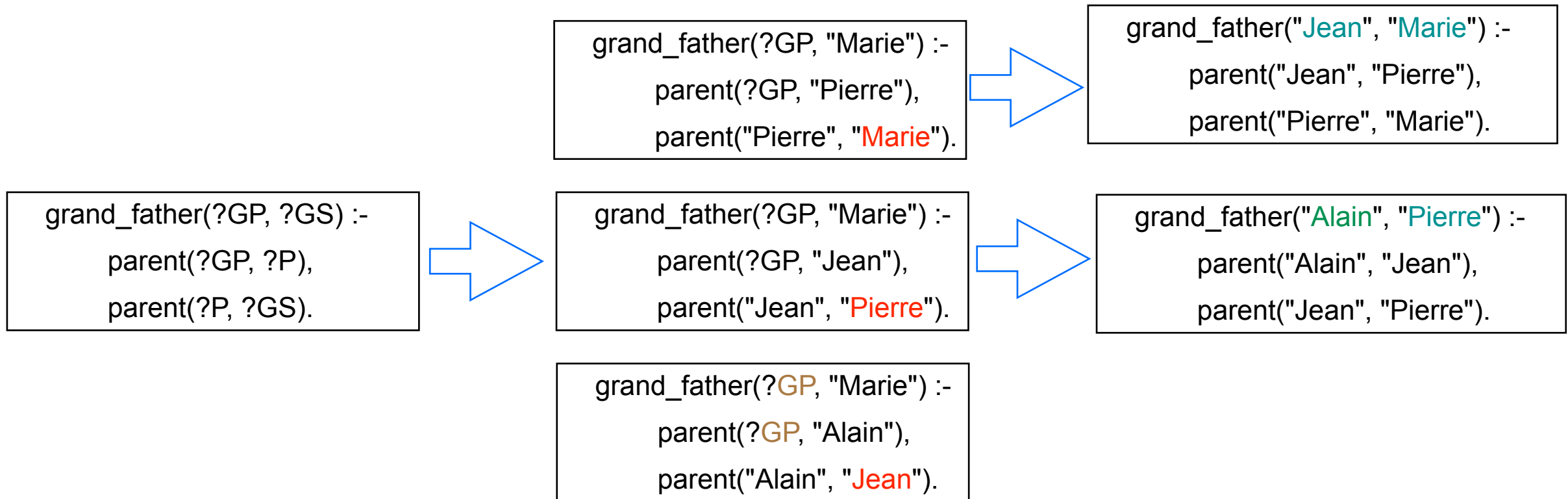
- We can then answer questions such as: *Who are the grand-fathers ?*



Prolog: Traversing a Graph II

```
parent("Pierre", "Marie"). Pierre is a parent of Marie
parent("Jean", "Pierre"). Jean is a parent of Pierre
parent("Alain", "Jean"). Alain is a parent of Jean
```

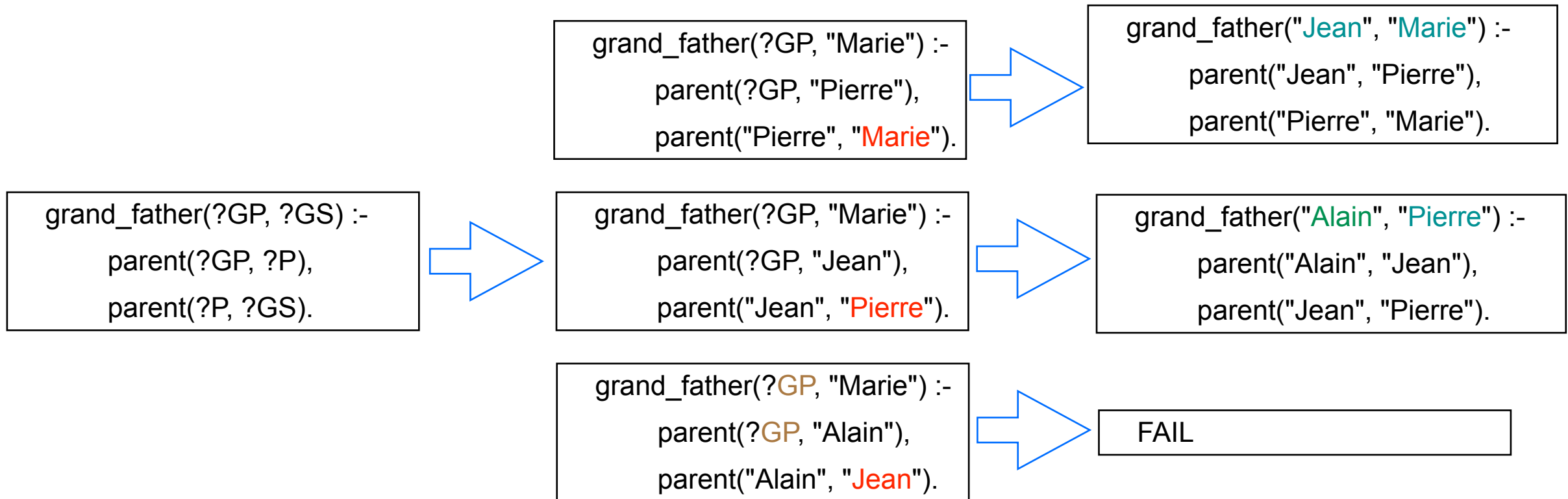
- We can then answer questions such as: *Who are the grand-fathers ?*



Prolog: Traversing a Graph II

```
parent("Pierre", "Marie"). Pierre is a parent of Marie
parent("Jean", "Pierre"). Jean is a parent of Pierre
parent("Alain", "Jean"). Alain is a parent of Jean
```

- We can then answer questions such as: *Who are the grand-fathers ?*



Comparison with LangGraph

1. Native Backtracking and Search

- Prolog: Built-in, mathematically sound backtracking mechanism
- LangGraph: Must explicitly implement backtracking, state management, and search strategies

2. Unified State Management

- Prolog: Natural unification and variable binding
- LangGraph: Requires custom state objects, explicit state passing, manual consistency checks

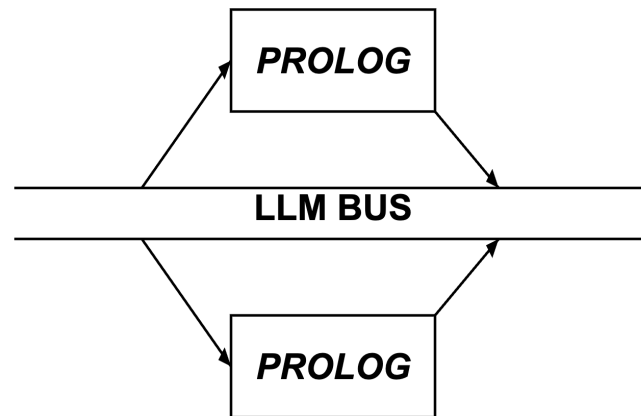
3. Formal Foundations

- Prolog: Based on formal logic, with proven properties
- LangGraph: Ad-hoc solutions for logical operations, conditional branching, path management

The AI Bus

Concept: Imagine an "AI Bus" inspired by message-passing systems (like in Erlang or Smalltalk).

LLMs, being stochastic agents, publish their outputs (data structures, observations, generated text, code snippets, etc.) onto this bus as messages.

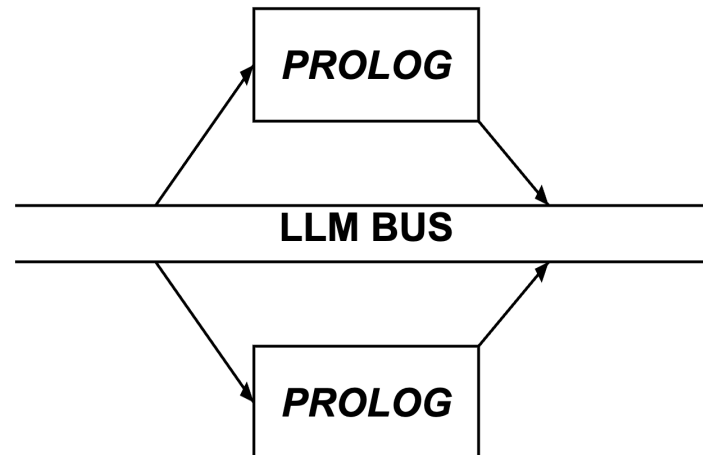


How Prolog Interacts with the AI Bus:

- **Prolog as the Consumer/Processor:** Prolog rules act as intelligent listeners on the AI Bus. They define predicates that specify the patterns and structures of messages they are interested in "catching" from the bus.
- **Pattern Matching and Unification:** Prolog's powerful pattern matching and unification capabilities are ideal for identifying and extracting relevant data structures from the diverse and potentially noisy stream of LLM outputs on the bus.
- **Rule-Based Reasoning:** Once a message is "caught" by a Prolog rule, the system can apply logical reasoning based on its knowledge base and the predicates defined. This allows for structured processing of the LLM's output.
- **Handling Stochasticity:** By using Prolog to process LLM outputs, the system can introduce a layer of structure and logic to handle the inherent stochasticity of LLMs. Prolog's backtracking allows exploring different interpretations or paths based on the data received from the bus.
- **State Management:** Prolog's natural approach to state management helps in maintaining the context and flow of information processed from the AI Bus, providing a more robust system than ad-hoc methods.
- **Enabling Complex Behaviors:** The interaction of different Prolog rules, triggered by messages on the bus, can lead to the emergence of complex agent behaviors beyond simple reactive responses.

AI Bus to handle stochasticity

In essence, the AI Bus provides a flexible communication layer for LLMs, while Prolog offers the structured reasoning and processing capabilities needed to make sense of and act upon the information flowing through the bus, mitigating the challenges posed by the stochastic nature of LLM outputs.



Comparison with LangGraph II

4. Scalability of Complexity

- Prolog: Complexity grows linearly with problem size
- LangGraph: Complexity grows exponentially with custom nodes, state handlers, conditional edges

5. Turing-Complete

- Prolog: Turing-complete language, can express any functions
- LangGraph: DSL, can only be extended through Python functions or new node implementation

6. Behavior

- Prolog: Can develop complex behaviors through rule interaction
- LangGraph: Behaviors limited to explicit graph definition

Testbed

We choose a dataset: GSM8k, which contains problems such as:

Problem: Mark has a garden with flowers. He planted plants of three different colors in it. Ten of them are yellow, and there are 80% more of those in purple. There are only 25% as many green flowers as there are yellow and purple flowers. How many flowers does Mark have in his garden?

Answer: So in total Mark has $28 + 7 = \langle\langle 28+7=35 \rangle\rangle 35$ plants in his garden.

We have both the Problem and its answer...

How we will proceed:

SYSTEM: You are an expert in solving problems extracted from a dataset. You will proceed step by step to solve the problems.

USER: Here is the problem to solve: Mark has a garden with flowers. He planted plants of three different colors in it. Ten of them are yellow, and there are 80% more of those in purple. There are only 25% as many green flowers as there are yellow and purple flowers. How many flowers does Mark have in his garden?

Generate 5 relevant questions to solve this problem.

USER: *Please answers the following question. Keep your response as concise as possible: How many yellow flowers does Mark have?*

ASSISTANT: Ten

USER: Think of another relevant question to ask?

ASSISTANT: What is the total number of purple flowers?

USER: Returns this question in a JSON list without dictionaries.

ASSISTANT: ["What is the total number of purple flowers?"]

USER: Please answers these questions as concisely as possible: What is the total number of purple flowers?

ASSISTANT: 16

Code Generation

USER: Provides a Python program to solve this problem.

ASSISTANT:

```
```python
yellow_flowers = 10
purple_flowers = yellow_flowers * (80/100 + 1)
green_flowers = (yellow_flowers + purple_flowers) * (25/100)
total_flowers = yellow_flowers + purple_flowers + green_flowers
(purple_flowers, total_flowers)
```
```

USER: Modify that code so that the final result is stored in a specific global variable named: `total`.

ASSISTANT:

```
```python
yellow_flowers = 10
purple_flowers = yellow_flowers * (80/100 + 1)
green_flowers = (yellow_flowers + purple_flowers) * (25/100)
total = yellow_flowers + purple_flowers + green_flowers
```
```

Why Prolog matters

The implicit graph is described with rules

```
analyse(?model, ?expected, ?problem, ?R) :-  
    ?system is 'You are an expert in solving GSM problems extracted from a dataset. You will proceed step by step to solve the problems. ',  
    ?prompt_inter is "Here is the problem to solve: " + ?problem,  
    ?prompt is ?prompt_inter + "\nThink of " + solutions + " initial questions with a numerical focus that could help solve the problem.",  
    get_enough_questions(?model, ?system, ?prompt, ?liste),  
    explore_level_one(?model, ?expected, ?system, ?prompt_inter, ?liste, ?R, 1),!.
```

Why Prolog matters

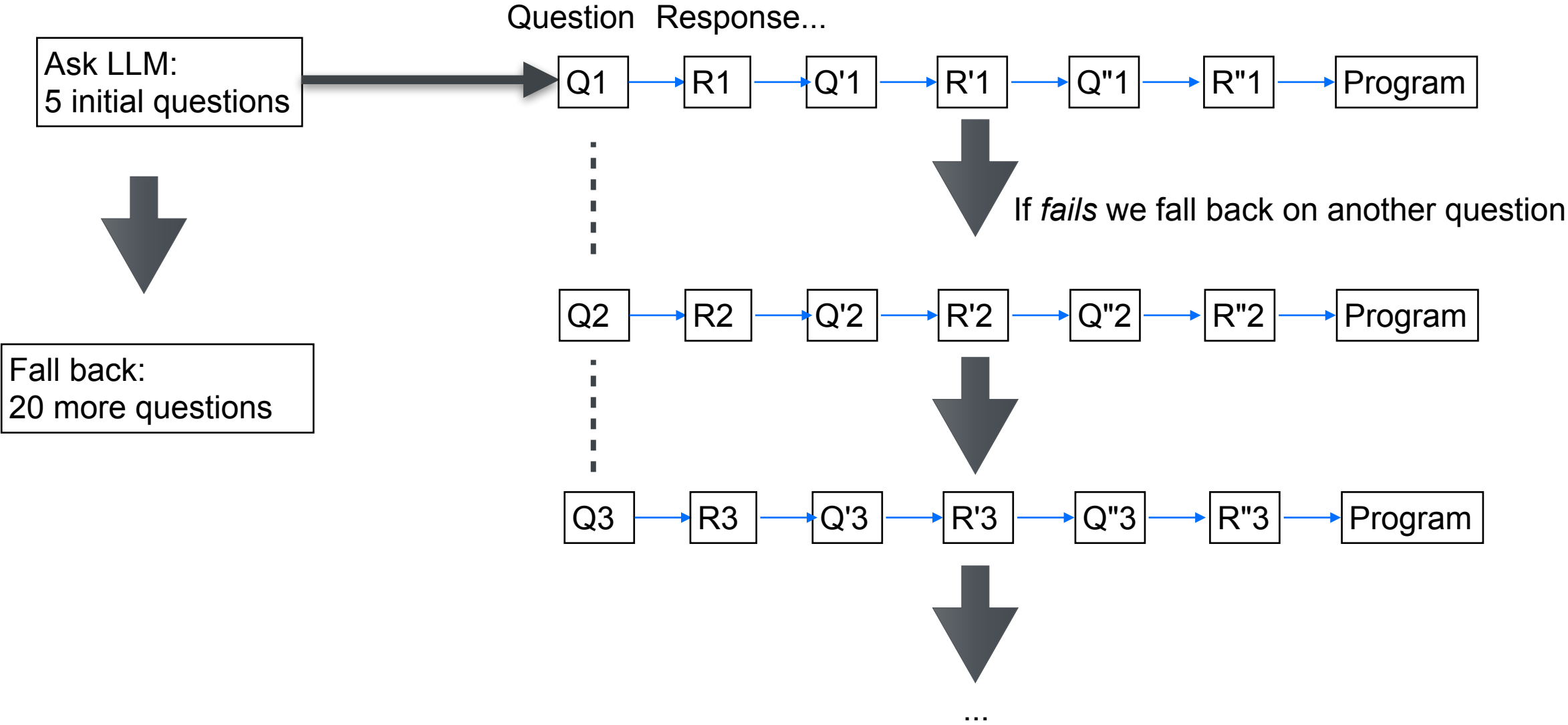
We can easily handle errors

```
get_enough_questions(?model, ?system, ?prompt, ?liste) :-  
    ?chat is chat_prompt_init(?model, ?system, ?prompt),  
    ?resultat is chat_prompt(?model, "Returns these questions as a flat JSON list (no dictionaries):", ?chat),  
    extract_vector(?resultat, solutions, ?liste),!. //The cut prevents the execution of the next rule  
  
//If extract_vector fails we try again...  
get_enough_questions(?model, ?system, ?prompt, ?liste) :-  
    println("Posing questions again, not enough questions produced before."),  
    get_enough_questions(?model, ?system, ?prompt, ?liste).
```

We can then execute the code

```
generatecode(?model, ?expected, ?chat, ?res) :-  
    ?r is chat_prompt(?model, "\nProvides a Python program to solve this problem.", ?chat),  
    ?chatcode is chat_prompt(?model, "\nModify that code so that the final result is stored in a global variable named: `total`.", ?r),  
    extract_code(?chatcode, ?code),  
    execute_code(?code, "total", ?result), //If it fails the whole branch fails  
    selection(?chatcode, ?result, ?expected, ?res).
```

Strategy



Questions and Code

The strategy can be summarize in three points:

1. Hierarchical Questioning

Uses LLM to first generate broad numerical questions, then systematically explores deeper with follow-up questions, mimicking human problem decomposition.

2. Two-Level Exploration Strategy

First level generates initial questions, second level attempts solutions for each question path, with controlled depth to prevent infinite exploration.

3. Code-Driven Verification

All proposed solutions must be expressed as executable Python code with results in a '**total**' variable, ensuring concrete, testable outcomes.

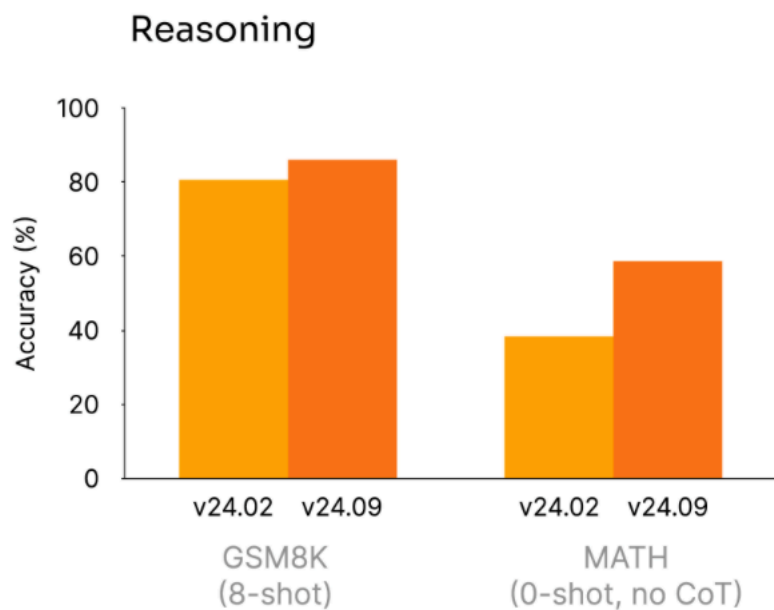
Code Generation

➡ Why choose a code generation approach?

1. LLMs are bad at math. Asking an LLM to do mathematical operations is usually not reliable as a strategy, especially with *large numbers*.
2. There is always a doubt on contamination. Most of the experiments with GSM8k consists of generating the right answer straight away, however if LLMs are bad at math, it put some doubts on the results.
 - a. Pure pattern matching. Apple¹ showed that a slight modification of the problem can lead to catastrophic outputs.
 - b. Code generation is clean. You have your numerical output without doing any string extraction.

Model

- We use *mistral small*, a 22B parameters model, on the GSM8K dataset.



Model Card

| | |
|--------------------|--------|
| parameters | 22.2B |
| context length | 131072 |
| embedding length | 6144 |
| quantization | Q4_0 |
| temperature | 0.95 |
| num_ctx(nb tokens) | 2000 |

Dataset: GSM8K (Grade School Math 8k)

8.5K high quality linguistically diverse grade school math word problems.

Created to support the task of question answering on basic mathematical problems that require multi-step reasoning.

- These problems take between 2 and 8 steps to solve.
- Solutions primarily involve performing a sequence of elementary calculations using basic arithmetic operations (+ − × ÷) to reach the final answer.
- A bright middle school student should be able to solve every problem: from the paper, "Problems require no concepts beyond the level of early Algebra, and the vast majority of problems can be solved without explicitly defining a variable."
- Solutions are provided in natural language, as opposed to pure math expressions. From the paper: "We believe this is the most generally useful data format, and we expect it to shed light on the properties of large language models' internal monologues"

For 7473 problems

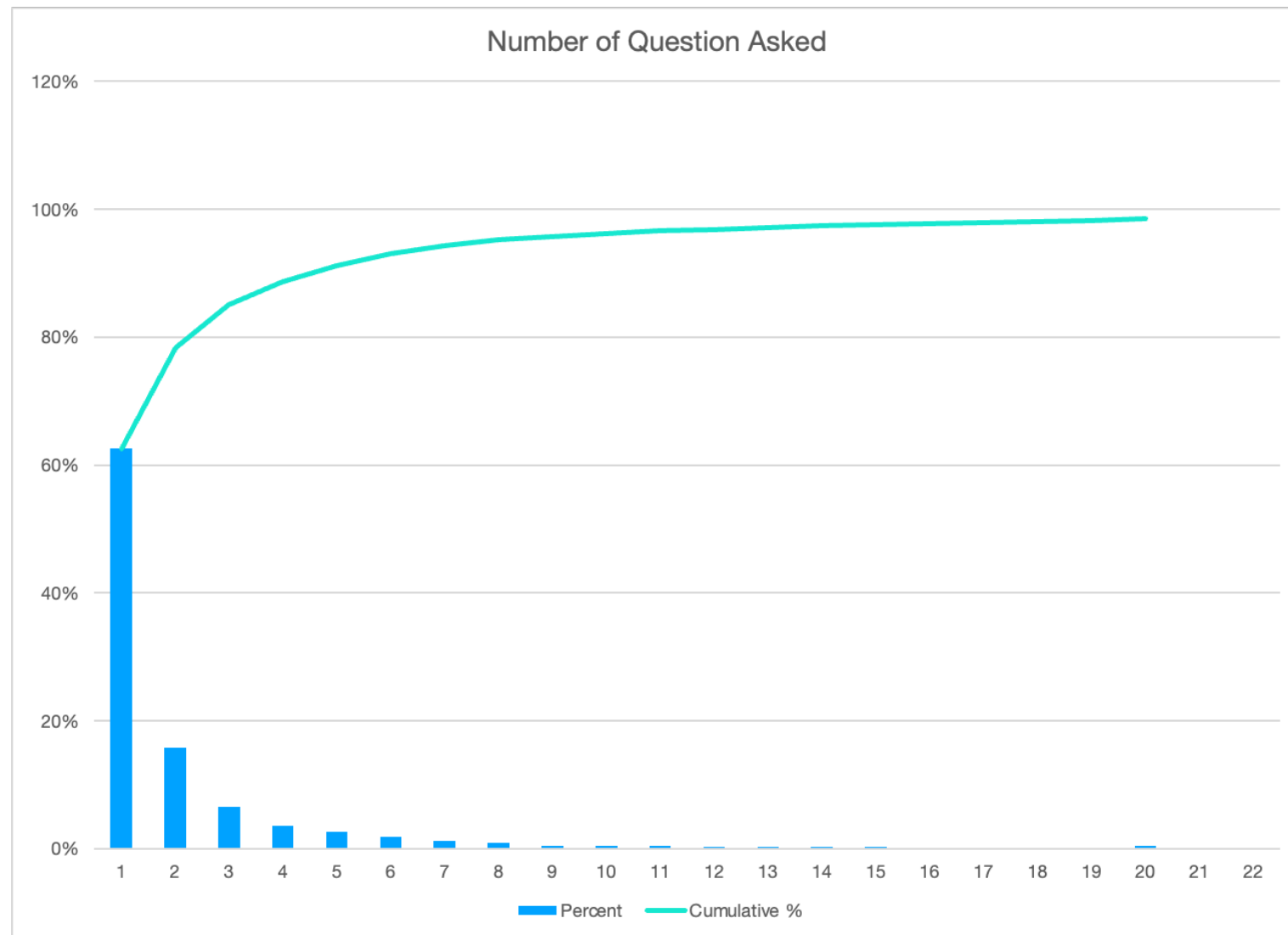
Number of attempts (pass@5 then pass@20)

| Attempts | Count |
|----------|-------|
| Fail | 109 |
| 1st | 4 675 |
| 2nd | 1 184 |
| 3rd | 494 |
| 4th | 270 |
| 5th | 192 |
| 6th | 143 |
| 7th | 95 |
| 8th | 66 |
| 9th | 37 |
| 10th | 36 |
| 11th | 29 |
| 12th | 19 |
| 13th | 19 |
| 14th | 17 |
| 15th | 19 |
| 16th | 11 |
| 17th | 15 |
| 18th | 7 |
| 19th | 8 |
| 20th+ | 28 |

| | Count | Percent |
|---------|-------|---------|
| Fails | 109 | 1.46% |
| Success | 7364 | 98.54% |
| 0 shot | 4675 | 62.56% |
| 5 shots | 6815 | 91.19% |
| 8 shots | 7119 | 95.26% |
| Avrg | 2.13 | |

Most LLMs experiments on datasets are done in multiple shots.

*You do **n** experiments (pass@**n**) and you check if the expected results show in the output.*



7473 problems

| | Count | Percent |
|---------|-------|---------|
| Success | 7364 | 98.54% |
| 0 shot | 4675 | 62.56% |
| 5 shots | 6815 | 91.19% |
| 8 shots | 7119 | 95.26% |

| Rank | Model | Accuracy↑ | Parameters
(Billion) | Extra
Training
Data | Paper | Code | Result | Year | Tags  |
|------|--|-----------|-------------------------|---------------------------|---|---|---|------|--|
| 1 | Qwen2-Math-72B-Instruct
(greedy) | 96.7 | 72 | ✓ | Qwen2 Technical Report |  |  | 2024 | |
| 2 | SFT-Mistral-7B
(Metamath, OVM, Smart Ensemble) | 96.4 | 7 | ✓ | | | | 2024 | |
| 3 | OpenMath2-Llama3.1-70B
(majority@256) | 96.0 | | ✓ | OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data |  |  | 2024 | |
| 4 | Jiutian-大模型 | 95.2 | 75 | × | | | | 2024 | |
| 5 | DAMOMath-7B
(MetaMath, OVM, BS, Ensemble) | 95.1 | 7 | ✓ | | | | 2024 | |
| 6 | Claude 3 Opus
(0-shot chain-of-thought) | 95 | | × | The Claude 3 Model Family: Opus, Sonnet, Haiku | |  | 2024 | <div>chain-of-thought</div> <div>zero-shot</div> |
| 7 | OpenMath2-Llama3.1-70B | 94.9 | | ✓ | OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data |  |  | 2024 | |
| 8 | GPT-4
(Teaching-Inspired) | 94.8 | | × | Teaching-Inspired Integrated Prompting Framework: A Novel Approach for Enhancing Reasoning in Large Language Models |  |  | 2024 | |

Prolog for Agent Implementation

❖ **Prolog-based Framework**

The proposed framework leverages Prolog's declarative nature and rule-based reasoning to efficiently implement intelligent agents.

❖ **Motivated by LLM Limitations**

This is motivated by the stochastic nature of large language models (LLMs), which need to be integrated into a structured system to provide their best outputs.

❖ **Question-First Architecture**

By transforming problem-solving into a question-first architecture, the system creates multiple potential solution paths before attempting solutions, ensuring concrete, testable results through enforced code generation and a robust verification system.

❖ **Future Developments**

Building on this foundation, future developments could explore integrating the framework with deep learning models, expanding the problem domains, and enhancing the user experience.

ADDENDUM

1. Two-Tier Efficiency Structure

- First pass: 5 questions for maximum efficiency
- Second pass: 20 questions for harder problems
- Resource-optimized escalation strategy

2. Question-First Architecture

- Transforms problem-solving into question generation
- Model acts as a question engine
- Creates multiple potential solution paths before attempting solutions

3. Structured Exploration

- Two-level exploration tree for each problem
- First level generates strategic questions
- Second level explores solutions within each question path

4. Enforced Code Generation

- Solutions must be executable Python code
- Results stored in specific 'total' variable
- Ensures concrete, testable solutions

5. Single Model Strategy

- Complete workflow with same LLM
- Strategy-focused rather than model-focused
- Cost-efficient approach to problem-solving

6. Failure-Embracing Framework

- Built-in expectation of initial failures
- Progressive scaling for difficult problems
- Systematic approach to retries

7. Verification System

- Executable code verification
- Multi-stage validation process
- Problem-specific result checking

```

if __name__ == '__main__':
    prompt = """You are an expert of the Tamgu programming language. Your goal is to answer questions about the Tamgu
        programming language based on the documentation pages"""

    continuation = f"""Given a textual input containing both a query and corresponding documentation pages,
        carefully review each documentation page to identify the correct response.
        Make sure to utilize only the definitions of the methods and types present in the provided documentation pages
        to ensure an accurate and contextually appropriate answer. Returns the response as a Markdown file:
    """

    prompt += continuation

    event = eventloop.Eventloop()

    # Each Node is a class that exposes an `execute` method.
    event.record_node("query", NodeQueryChromaTXT("query", ["explain"]))
    event.record_node("explain", NodeRawOllama("preprompt", ["log"], "mixtral", prompt, temperature = 0.1))
    event.record_node("log", NodeLog("logs.txt", []))

    event.post(Message("query", 'How do you implement the "in" operator in Tamgu for frames?'))
    event.main_loop()

```

In summary

1. Native Backtracking and Search

- Result: Cleaner, more reliable code for complex exploration patterns

2. Unified State Management

- Result: Less code, fewer bugs, more robust state handling

3. Formal Foundations

- Result: More predictable behavior at scale, better theoretical guarantees

4. Scalability of Complexity

- Result: More maintainable systems at scale

execute_code

```
// Function to execute Python code and return the result
// v will be unified with the result of the execution
function execute_code(string code, string outvar, ?_ v) {
    // We need a variable to handle the Python interpreter
    python p;

    string py_err;
    try {
        v = p.run(code, outvar, 2);
        return true;
    }
    catch(py_err) {
        v = py_err;
        return false;
    }
}
```

- If `run` fails, then an exception is raised and `execute\_code` will return *false*. `v` will then be unified with the error message
- If `run` succeeds then `v` will be unified with the result.
- *outvar* is the name of the variable in the Python code that stores the final result.
- 2 is a timer to stop the execution if it takes too long. *In some cases, the LLM might generate infinite loops.*

CRITIC: LARGE LANGUAGE MODELS CAN SELF- CORRECT WITH TOOL-INTERACTIVE CRITIQUING (Zhibin Gou et al.)

ICLR 2024

<https://arxiv.org/pdf/2305.11738>

ABSTRACT

Recent developments in large language models (LLMs) have been impressive. However, these models sometimes show inconsistencies and problematic behaviors such as hallucinating facts, generating flawed code, or creating offensive and toxic content. Unlike these models, humans typically utilize external tools to check and refine their initial content, like using a search engine for fact-checking, or a code interpreter for debugging. Inspired by this observation, we introduce a framework called CRITIC that allows LLMs, which are essentially “black boxes” to validate and progressively amend their own outputs in a manner similar to human interaction with tools. More specifically, starting with an initial output, CRITIC interacts with appropriate tools to evaluate certain aspects of the text, then revises the output based on the feedback obtained during this validation process. Comprehensive evaluations involving free-form question answering, mathematical program synthesis, and toxicity reduction demonstrate that CRITIC consistently enhances the performance of LLMs. Meanwhile, our research highlights the crucial importance of external feedback in promoting the ongoing self-improvement of LLMs¹.

SELF-REFINE: Iterative Refinement with Self-Feedback (Aman Madaan et al.)

<https://arxiv.org/pdf/2303.17651>

ABSTRACT

Like humans, large language models (LLMs) do not always generate the best output on their first try. Motivated by how humans refine their written text, we introduce SELF-REFINE, an approach for improving initial outputs from LLMs through iterative feedback and refinement. The main idea is to generate an initial output using an LLM; then, the same LLM provides *feedback* for its output and uses it to *refine* itself, iteratively. SELF-REFINE does not require any supervised training data, additional training, or reinforcement learning, and instead uses a single LLM as the generator, refiner and the feedback provider. We evaluate SELF-REFINE across 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, using state-of-the-art (GPT-3.5 and GPT-4) LLMs. Across all evaluated tasks, outputs generated with SELF-REFINE are preferred by humans and automatic metrics over those generated with the same LLM using conventional one-step generation, improving by ~20% absolute on average in task performance. Our work demonstrates that even state-of-the-art LLMs like GPT-4 can be further improved at test-time using our simple, standalone approach.¹

Reflexion: Language Agents with Verbal Reinforcement Learning (Noah Shinn et al.)

<https://arxiv.org/pdf/2303.11366>

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead by providing linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) of feedback sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at <https://github.com/noahshinn024/reflexion>.

Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations? (Zorik Gekhman et al.)

<https://arxiv.org/pdf/2405.05904>

Abstract

When large language models are aligned via supervised fine-tuning, they may encounter new factual information that was not acquired through pre-training. It is often conjectured that this can teach the model the behavior of *hallucinating* factually incorrect responses, as the model is trained to generate facts that are grounded in its pre-existing knowledge. In this work, we study the impact of such exposure to new knowledge on the capability of the fine-tuned model to maintain its pre-existing knowledge. To this end, we design a controlled setup, focused on closed-book QA, where we vary the proportion of the fine-tuning examples that introduce new knowledge. We demonstrate that large language models struggle to acquire new factual knowledge through fine-tuning, as fine-tuning examples that introduce new knowledge are learned significantly slower than those consistent with the model's knowledge. However, we also find that as the examples of new knowledge are eventually learned, they linearly increase the model's tendency to hallucinate. Taken together, our results highlight the risk in introducing new factual knowledge through fine-tuning, and support the view that large language models mostly acquire factual knowledge through pre-training, whereas fine-tuning teaches them to use it more efficiently.

GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models

<https://arxiv.org/pdf/2410.05229>

Abstract

Recent advancements in Large Language Models (LLMs) have sparked interest in their formal reasoning capabilities, particularly in mathematics. The GSM8K benchmark is widely used to assess the mathematical reasoning of models on grade-school-level questions. While the performance of LLMs on GSM8K has significantly improved in recent years, it remains unclear whether their mathematical reasoning capabilities have genuinely advanced, raising questions about the reliability of the reported metrics. To address these concerns, we conduct a large-scale study on several state-of-the-art open and closed models. To overcome the limitations of existing evaluations, we introduce GSM-Symbolic, an improved benchmark created from symbolic templates that allow for the generation of a diverse set of questions. GSM-Symbolic enables more controllable evaluations, providing key insights and more reliable metrics for measuring the reasoning capabilities of models. Our findings reveal that LLMs exhibit noticeable variance when responding to different instantiations of the same question. Specifically, the performance of all models declines when only the numerical values in the question are altered in the GSM-Symbolic benchmark. Furthermore, we investigate the fragility of mathematical reasoning in these models and demonstrate that their performance significantly deteriorates as the number of clauses in a question increases. We hypothesize that this decline is due to the fact that current LLMs are not capable of genuine logical reasoning; instead, they attempt to replicate the reasoning steps observed in their training data. When we add a single clause that appears relevant to the question, we observe significant performance drops (up to 65%) across all state-of-the-art models, even though the added clause does not contribute to the reasoning chain needed to reach the final answer. Overall, our work provides a more nuanced understanding of LLMs' capabilities and limitations in mathematical reasoning.

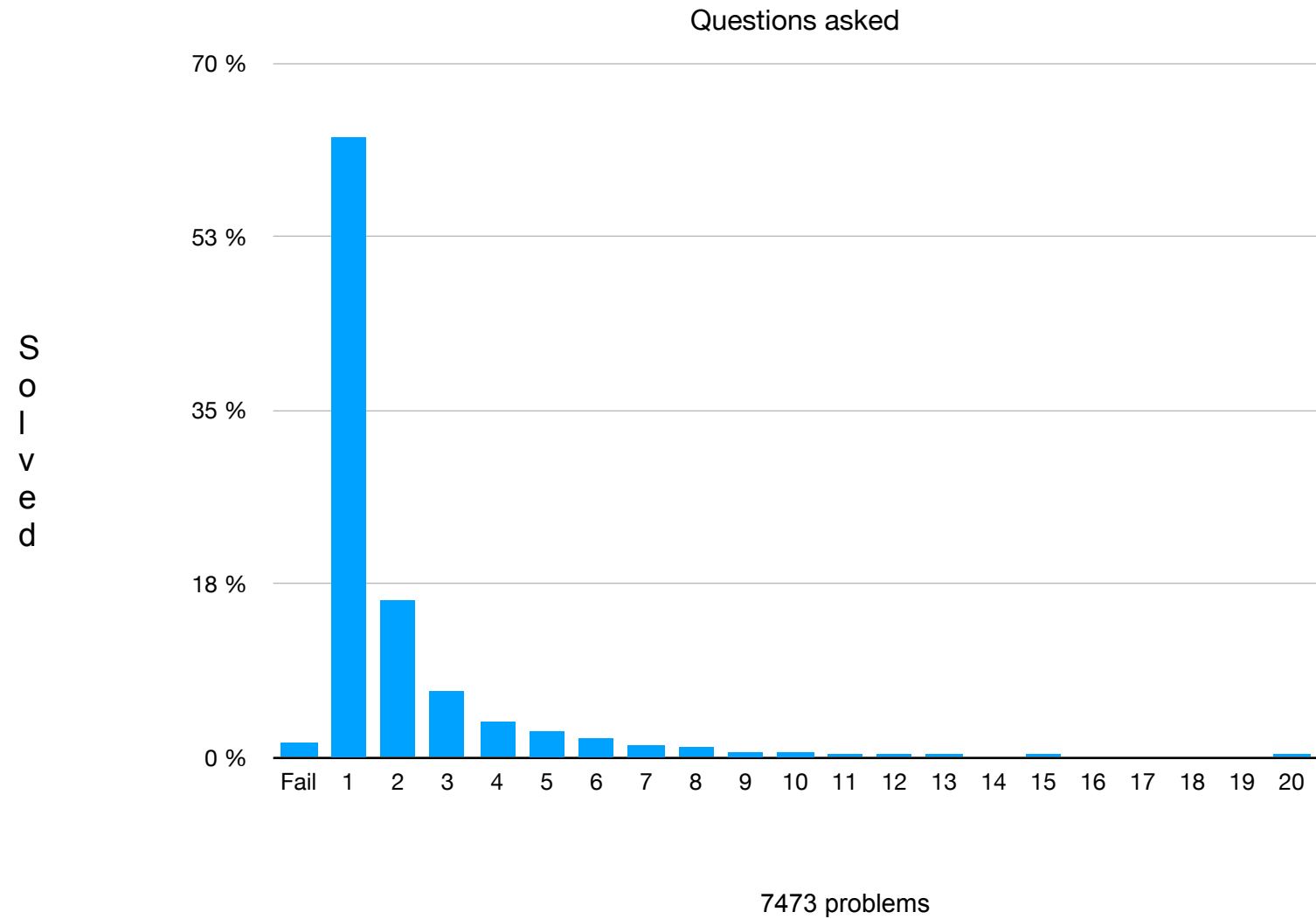
Automated Evaluation of Retrieval-Augmented Language Models with Task-Specific Exam Generation (Gauthier et al.)

<https://arxiv.org/abs/2405.13622>

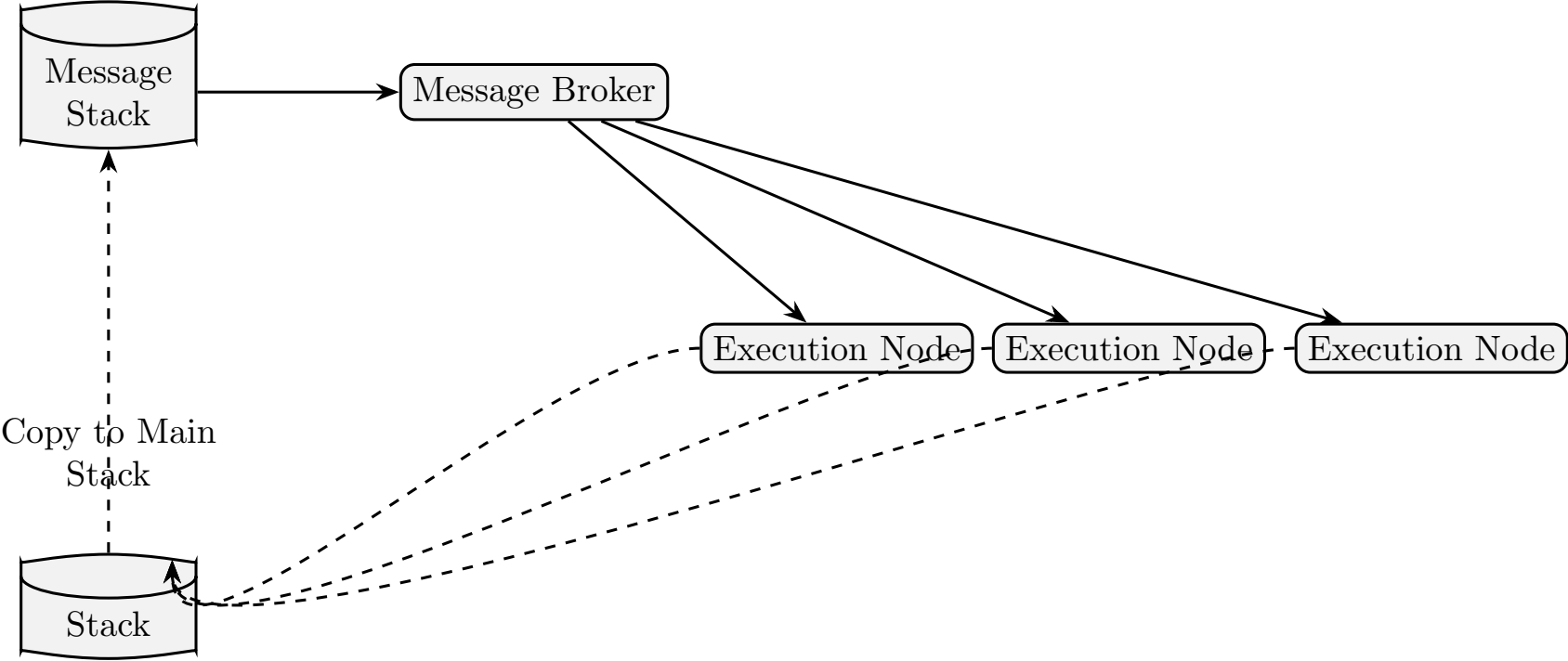
Abstract

We propose a new method to measure the task-specific accuracy of Retrieval-Augmented Large Language Models (RAG). Evaluation is performed by scoring the RAG on an automatically-generated synthetic exam composed of multiple choice questions based on the corpus of documents associated with the task. Our method is an automated, cost-efficient, interpretable, and robust strategy to select the optimal components for a RAG system. We leverage Item Response Theory (IRT) to estimate the quality of an exam and its informativeness on task-specific accuracy. IRT also provides a natural way to iteratively improve the exam by eliminating the exam questions that are not sufficiently informative about a model's ability. We demonstrate our approach on four new open-ended Question-Answering tasks based on Arxiv abstracts, StackExchange questions, AWS DevOps troubleshooting guides, and SEC filings. In addition, our experiments reveal more general insights into factors impacting RAG performance like size, retrieval mechanism, prompting and fine-tuning. Most notably, our findings show that choosing the right retrieval algorithms often leads to bigger performance gains than simply using a larger language model.

<https://huggingface.co/blog/evaluation-structured-outputs>



1. Agent as a Message Architecture



GSM8k

Problem: Mark has a garden with flowers. He planted plants of three different colors in it. Ten of them are yellow, and there are 80% more of those in purple. There are only 25% as many green flowers as there are yellow and purple flowers. How many flowers does Mark have in his garden?

q1: "How many more purple flowers are there than yellow flowers?"

r1: There are $80/100 * 10 = \ll 80/100 * 10 = 8 \gg 8$ more purple flowers than yellow flowers.

q2: How many purple flowers are there?

r2: So in Mark's garden, there are $10 + 8 = \ll 10 + 8 = 18 \gg 18$ purple flowers.

q3: How many flowers are there in total?

r3: Purple and yellow flowers sum up to $10 + 18 = \ll 10 + 18 = 28 \gg 28$ flowers.

q4: How many green flowers are there?

r4: That means in Mark's garden there are $25/100 * 28 = \ll 25/100 * 28 = 7 \gg 7$ green flowers.

Q last: How many plants does Mark have in his garden?

R last: So in total Mark has $28 + 7 = \ll 28 + 7 = 35 \gg 35$ plants in his garden.

Chain of Thought and Thinking



- CoT is now universally accepted as the basic strategy to use LLM.
 - You ask the model to work step by step
- *Thinking* is also a very common strategy, which is used by a number of models.
 - It is part of the reflexive strategy of O1 for instance.

We will propose a new strategy, which borrows from both approaches, but with a *twist*.

https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

<https://openai.com/index/introducing-openai-o1-preview/>